VIT BHOPAL

A Capstone Project for OSS NGMC Course

Student Name : Ashi Gupta

Registration No : 24BAI10954

Course : Open Source Software (CSE0002)

Slot : B22

Faculty : Pradeep Kumar Mishra

Date of submission : 31st March 2026

<u>The Open Source Audit</u> : *Python Programming Language*

1. Introduction

A paradigm shift in software distribution, collaborative engineering, and intellectual property is represented by open-source software (OSS). Open-source development democratizes access by enabling anyone to examine, alter, and distribute a program's underlying architecture, in contrast to proprietary models that protect source code as a trade secret. OSS is the cornerstone of digital infrastructure in contemporary computing. Open-source projects supply the fundamental layers that modern technology runs on, from the servers that handle internet traffic to the hypervisors that oversee cloud data centers. Rapid innovation, peer-reviewed security, and a globalized approach to problem-solving that cuts across corporate boundaries are all made possible by OSS's inherent transparency.

"The development and distribution of technology has been revolutionized by open-source software. Open-source software permits anyone to view, alter, and distribute code without restriction, in contrast to proprietary software, where source code is concealed and managed by businesses. Global cooperation, enhanced security, and quick innovation have all resulted from this strategy."

The Python programming language stands out as a crucial topic for academic audit among the extensive ecosystem of open-source projects. Python was selected for this capstone analysis because of its rapid expansion, widespread use in cutting-edge domains like artificial intelligence, and distinctive governance model. Python is a prime example of the success of community-driven development. It demonstrates that an open-source tool can compete with, and frequently outperform, heavily funded proprietary alternatives in terms of performance, adoption, and ecosystem maturity by bridging the gap between low-level system operations and high-level human logic.

Python, a high-level programming language renowned for its ease of use and adaptability, is one of the most significant open-source technologies. Python's readability and extensive ecosystems have made it one of the most popular languages in the world.

PART A : Origin and Philosophy

A1. Origin Story

Python's origins can be traced back to the real-world problems with systems programming in the late 1980s. Python was developed as a "hobby" programming project to keep Dutch programmer Guido van Rossum busy during the Christmas week of 1989 at the Centrum Wiskunde & Informatica (CWI) in the Netherlands. Nonetheless, the language's intellectual underpinnings were a purposeful reaction to the shortcomings of the tools at the time, mainly C and the ABC programming language.

Guido van Rossum developed Python in 1989, and it was first made available in 1991.

Historical Background

In the late 1980s:

• Low-level languages like C dominated programming.

• A lot of code was needed for software development.

• Maintenance and debugging were challenging.

There was no language that combined the following:

• Usability

• Strength

• Adaptability

During that era, developing administrative utilities for the Amoeba distributed operating system in C was a laborious process. C demanded meticulous memory management, explicit type declarations, and verbose syntax, which slowed down rapid prototyping and conceptual testing. Van Rossum aimed to develop a language that combined the system-level features needed to successfully interface with Amoeba and other operating systems with the structural clarity of ABC, a language intended for teaching and usability.

Python was purposefully created with the idea of code readability in mind. Programmers were forced to write visually clean code because code blocks were indicated by significant whitespace (indentation) rather than the curly braces {} that are common in languages derived from C. This was an engineering philosophy rather than just an aesthetic decision. Software is read far more frequently than it is written. Python lessened the cognitive burden on developers by reducing syntactic noise, freeing them up to concentrate on algorithmic logic instead of language-specific boilerplate.

An important decision was made in 1991 to make Python available as open-source software, beginning with version 0.9.0 on the alt.sources newsgroup. Releasing the source code made it possible for a small group of early adopters to compile the interpreter on different UNIX systems during a time when software monetization primarily depended on licensing proprietary binaries. Rapid porting to various architectures, instant peer feedback, and community-contributed bug fixes were made possible by this openness. Van Rossum took on the humorous moniker "Benevolent Dictator for Life" (BDFL), directing the language's development via community consensus (through Python Enhancement Proposals, or PEPs) while maintaining final architectural authority. This model continued the project until he switched to a steering council model in 2018.

A2. License Analysis

Key Characteristics

• Allows modification
• Allows redistribution

• Allows commercial use
• Minimal restrictions

This encourages adoption in industries.

The Python Software Foundation (PSF) License is the legal framework that governs Python. Navigating the complicated terrain of open-source legalities is necessary to comprehend this license. A permissive free software license is the PSF License. Although its requirements are clearly different, it is compatible with the GNU General Public License (GPL) and is widely acknowledged as open source by the Open Source Initiative (OSI).

1. We must first compare the PSF License to the four fundamental freedoms of the Free Software Foundation (FSF) in order to evaluate it:
2. The freedom to run the program for any purpose (Freedom 0): Python does not impose any limitations on the location or manner in which the interpreter can be used, whether for commercial or academic purposes.
3. The freedom to learn how the program operates and modify it to perform your computing as you see fit (Freedom 1): The standard CPython implementation's C source code is fully open and documented.
4. The freedom to redistribute copies in order to assist others (Freedom 2): Python source code and binaries are freely shared.
5. Freedom 3: The ability to share copies of your altered versions with others

Importantly, businesses are legally allowed to alter the Python source code, compile it, and market it as a component of a closed-source, proprietary product without being compelled to make their changes public because the PSF License is permissive rather than a strict copyleft license (like the GPL). This is a purposeful design decision to promote the highest level of corporate adoption.

License Comparison:

- A robust copyleft license is the GPL (GNU General Public License). Your altered version of GPL software must likewise be GPL if you distribute it. It puts the code's unending freedom ahead of the developer's ability to close it.
- The MIT license is very lenient. As long as the original copyright notice is included, you can do as you please.
- Similar to MIT and BSD, the PSF License permits proprietary derivatives but contains special provisions pertaining to Python's past copyright holders (CWI, CNRI, and the PSF) to guarantee a clean chain of intellectual property custody.

Python is a perfect example of the difference between "free as in beer" (gratis) and "free as in freedom" (libre). An unactivated version of WinRAR is an example of a proprietary freeware program that is "free as in beer"—you don't have to pay for it, but you can't comprehend or modify its workings. The definition of Python is "free as in freedom." The user's inalienable right to dissect, rebuild, and distribute the technology is its true value; its lack of a price tag is only a consequence of its liberal licensing.

A3. Ethics of Open Source

Do All Programs Need to Be Open Source?

Benefits -

Transparency → It's easy to find bugs
Security: Public audits can increase confidence.
Innovation → Developers enhance already-existing tools

Drawbacks -

Developers' financial difficulties
Danger of abuse
In certain projects, a lack of accountability

Significant ethical concerns about labor and intellectual property are brought up by the widespread use of open-source software. A radical viewpoint, supported by Richard Stallman, contends that all software ought to be free and that it is fundamentally oppressive to limit a user's ability to alter the tools they use. According to this perspective, proprietary software establishes an unethical power dynamic in which the user is under the developer's control. The pragmatic argument, on the other hand, argues that large-scale technological advancement would stall without the assurance of a direct financial return and supports proprietary software as a necessary mechanism to incentivize capital-intensive research and development.

There is a lot of discussion about the morality of businesses making money off of open-source software like Python. On top of Python, multinational firms construct infrastructures worth billions of dollars, frequently without compensating the core language's developers. Is this morally right? It is lawful under the conditions of the PSF license. It becomes problematic from an ethical standpoint if it results in a "tragedy of the commons," in which corporate entities take enormous value from the ecosystem without giving back (through funding, bug fixes, or dedicated developer time), ultimately exhausting the volunteer maintainers. Nonetheless, many contend that corporate adoption promotes standardization, validates the software, and ultimately results in mutually beneficial partnerships whereby corporations support the foundations that uphold the language.

The idea of "standing on the shoulders of giants" is Python's moral and technical core. The decades of systems engineering that went into creating C compilers and POSIX-compliant operating systems were crucial to the development of Python, which was written in C. Python, on the other hand, is the giant that modern developers and data scientists rely on. The unpaid, open-source work of tens of thousands of people who created the hardware interfaces, C-bindings, mathematical libraries, and syntactic sugar that enable such quick development is utilized when an engineer trains a neural network in ten lines of Python. Understanding the ethics of open source requires an awareness of this continuous chain of cooperative human effort.

PART B : Linux Footprint

Python's footprint was directly examined on a Debian-based Linux environment using standard command-line interfaces to see how it integrated with the operating system in order to perform a thorough technical audit.
Python and Linux have a very mutually beneficial relationship. Python is regarded as a crucial system dependency rather than just a user-space application in the majority of contemporary Linux distributions.

Installation and Confirmation

Because many OS-level utilities (like the aptpackage manager itself) rely on Python scripts, Python 3 is usually pre-installed on a standard Debian/Ubuntu system. The standard package manager was utilized to make sure the environment was current:
Bash
The following commands were run in order to confirm the installation and find the binary execution path:
Bash

Locations of Files and OS Integration

The primary executable is located in /usr/bin/, the standard directory for user commands, according to the which command. But Python's influence goes farther. Third-party packages installed globally via pip or apt are usually located in /usr/local/lib/python3.x/dist-packages/, whereas the standard library and core modules are kept in /usr/lib/python3.x/.

Permissions for Users

The executable's status on the system can be found by looking at its permissions:

Bash

The output -rwxr-xr-x shows that the file is owned by the root user, who also has read, write, and execute rights. Read and execute permissions (r-x) are granted to standard users, such as the student account. Linux's core interpreter binary cannot be maliciously altered without administrative (sudo) escalation, despite users being able to run Python scripts freely. This is a fundamental security posture.

Service Architecture

Unlike web servers (Apache, Nginx) or database engines (PostgreSQL), By default, Python does not operate as a "service" or continuous background daemon. Python is an interpreter. To invoke it, read a script, carry out the instructions in a linear fashion, and then end, an active process is needed. The interpreter itself is still a temporary execution engine rather than a resident service, even though Python can be used to create continuous background services (managed via systemd).

Update Management

The package maintainers of the Linux distribution are responsible for conservatively managing updates to the core Python interpreter (via apt). This keeps important system scripts that rely on a particular version of Python from being broken by updates. To avoid dependency conflicts between various projects and the host operating system, pip (Python Package Installer) is used to manage user-level packages and libraries, ideally within isolated Virtual Environments (venv).

SCRIPT 1 - System Identity Report

Description:

Creates a thorough snapshot of the Linux system that is being used. Key system data, such as the hostname, OS version, kernel details, current user, system uptime, CPU architecture, and available memory, are gathered and shown. This script establishes the environment in which Python, an open-source program, functions as an orientation tool.

Key Concepts Used:

- Environment variables ($USER, $HOSTNAME)

- Command substitution ($(...))

- Built-in commands: uname, uptime, free, lscpu

- Formatted output using echo

Run Command:

bash

./scripts/script1.sh

```
Ashi@LAPTOP-SVRL9VA8:/mnt/c/Users/panig/OneDrive/Desktop/oss-audit-project$ bash script1_system_identity.sh
============================================================
        OPEN SOURCE AUDIT - SYSTEM IDENTITY REPORT
============================================================

  Student  : Ashi
  Reg No   : 24BAI10954
  Project  : Python

------------------------------------------------------------
  SYSTEM INFORMATION
------------------------------------------------------------
  Distribution : Ubuntu 24.04.4 LTS
  Kernel       : 6.6.87.2-microsoft-standard-WSL2
  User         : Ashi
  Home Dir     : /home/Ashi
  Uptime       : up 18 minutes
  Date         : Monday, 30 March 2026
  Time         : 17:14:51

------------------------------------------------------------
  LICENSE NOTE
------------------------------------------------------------
  This operating system (Linux kernel) is covered by:
  GNU General Public License v2 (GPL v2)

  This means you are free to: run, study, share, and
  modify it. Any modifications you distribute must also
```

SCRIPT 2 - FOSS Package Inspector (Python)

Description:

Examines the host system's Python installation from the standpoint of open-source software. The installed version of Python, its binary path, and the standard library's location are all identified. Additionally, it uses pip3 to list the third-party packages that are currently installed and verifies if Python is listed as a recognized package by the system's package manager (dpkg or rpm). As a FOSS artifact, Python is directly audited by this script.

Key Concepts Used:

- Command substitution and piping (|)

- Conditional checks with if/else

- Package manager queries (dpkg -l, pip3 list)

- String formatting and tabular output

Run Command:

bash

./scripts/script2.sh

SCRIPT 3 - Disk and Permission Auditor

Description:

Carries out a dual audit of the project directory's file permissions and disk usage. After reporting the filesystem's total, used, and available disk space, it recursively examines every file in the scripts/folder and uses stat and ls -l to display each file's size, ownership, and permission bits. It highlights the significance of appropriate file permission management in Linux by flagging any script that does not have execute permission.

Key Concepts Used:

- df for disk usage reporting

- stat and ls -l for permission inspection

- for loops to iterate over files

- Conditional logic to flag permission issues

- String comparison in Bash

Run Command:

bash

./scripts/script3.sh

SCRIPT 4 - Log File AnalyzeR

Description:

Mimics a small-scale log analysis tool. The script looks for entries that contain keywords like error, warning, failed, and Python by reading from /var/log/syslog (or a generated mock log if system logs are unavailable). Each keyword's occurrences are counted, matching lines are shown, and results are summarized. This is similar to real-world DevOps tasks where system logs are used to monitor Python-based tools.

Key Concepts Used:

- File reading and text processing with grep, awk, wc

- Keyword-based pattern matching

- Arrays in Bash

- while read loop for line-by-line file processing

- Conditional existence check for log files (-f flag)

Run Command:

bash

./scripts/[script4.sh]

```
==========================================================
        LOG FILE ANALYZER
==========================================================
 Log File :
 Keyword  : error
----------------------------------------------------------
 Error: No log file specified.
 Usage: script4_log_analyzer.sh <logfile> [keyword]
 Example: script4_log_analyzer.sh /var/log/syslog error
```

SCRIPT 5 - Open Source Manifesto Generator

Description:

Uses Python as a case study to create a formatted plain-text document called a "Open Source Manifesto" that explains the fundamentals of open-source software. The project title, student information, Python license details, OSS principles (freedom to run, study, redistribute, and improve), and a closing statement are all dynamically written by the script to output/manifesto.txt. It shows how to create, write, and redirect output in Bash.

Key Concepts Used:

- Output redirection (>, >>)

- Multi-line text generation using cat <<EOF

- Date and time stamping with date

- File I/O — creating and writing to files

- Variables for dynamic content injection

Run Command:

bash

./scripts/[script5.sh]

The generated manifesto will be saved to:

output/manifesto.txt

```
    THE OPEN SOURCE MANIFESTO GENERATOR
============================================================

  This script will generate your personal open source
  philosophy statement based on your answers.
  The manifesto will be saved to: manifesto_Ashi.txt

------------------------------------------------------------
  Please answer the following three questions:
------------------------------------------------------------

  1. Name one open-source tool you use every day: python

  2. In one word, what does 'freedom' mean to you? choice

  3. Name one thing you would build and share freely: an education app

------------------------------------------------------------
  Generating your manifesto...
------------------------------------------------------------

  Manifesto saved to: manifesto_Ashi.txt

============================================================
  YOUR MANIFESTO:
============================================================

============================================================
             MY OPEN SOURCE MANIFESTO
============================================================

  Author  : Ashi
  Date    : 30 March 2026

------------------------------------------------------------

  I believe that software should be free - not as in free beer,
  but free as in choice. Every day, Every day, I rely on python, a tool
  built by people who chose to share their work with the world
  rather than lock it away. That choice made me better at what I do.

  Open source is not just a licensing model. It is a philosophy
  that says knowledge grows when it is shared. The developers who
  built Linux, python, Python, and Git. Ashidid not hoard their
  work - they gave it away, and the world built on top of it.

  I commit to continuing that tradition. I will build an education app and
  share it freely, because I understand that I am standing on the
  shoulders of every developer who shared their code before me.
  Open source is how civilisation builds software together.

------------------------------------------------------------
  Signed: Ashi | 30 March 2026
============================================================
```

PART C : FOSS Ecosystem

Python’s extreme power reside in the sprawling Free and Open Source Software (FOSS) ecosystem that surrounds it , it does not only reside on its syntax.

Dependencies and Package Management

PyPI (the Python Package Index) and pip, the CLI tool that goes with it, are essential to this ecosystem. Hundreds of thousands of third-party packages are hosted on PyPI. When a developer needs a particular feature, such as matrix multiplication or cryptographic hashing, they declare a dependency rather than writing it from scratch.

DEPENDENCIES-

Python relies on:

- C compiler (GCC)
- System Libraries
- Package managers

Major Libraries

Highly optimized, open-source libraries that frequently serve as Python wrappers around high-performance C or C++ codebases dominate the ecosystem:

- **NumPy & Pandas:** The fundamental data manipulation libraries, NumPy and Pandas, offer quick, memory-efficient multi-dimensional arrays and dataframes.
- **TensorFlow & PyTorch:** The engines behind the current artificial intelligence boom are TensorFlow and PyTorch, which are fully open-sourced by Google and Meta, respectively, and provide unmatched access to deep learning architectures.
- **Requests & Beautiful Soup:** Industry standards for HTML parsing and HTTP protocol interaction, which serve as the foundation for automated data extraction.

Ecosystem Strength:

Python Ecosystem includes :

- Package Repository
- Thousands of libraries

Examples:

- Numpy (math)
- Pandas (data analysis)
- Matplotlib (visuals)

Real-World Applications

The versatility of this ecosystem allows Python to dominate multiple engineering domains. In my own practical experience developing automated AI agents and complex web crawling utilities (utilizing tools like the Firecrawl framework), Python's ecosystem proved indispensable. The ability to seamlessly integrate a natural language processing model with a headless browser automation script:all within a unified, readable codebase:demonstrates Python's utility in building intelligent, autonomous systems. Beyond AI, Python powers massive web applications (via Django and FastAPI) and serves as the primary scripting language for DevOps automation and cloud infrastructure management.

Community and Role

This ecosystem's adaptability enables Python to rule a variety of engineering fields. Python's ecosystem proved invaluable in my own real-world experience creating sophisticated web crawling tools and automated AI agents (using tools like the Firecrawl framework). Python's usefulness in creating intelligent, autonomous systems is demonstrated by the ability to

smoothly combine a headless browser automation script with a natural language processing model—all within a single, readable codebase. Beyond AI, Python is the main scripting language for cloud infrastructure management and DevOps automation, and it powers large web applications (through Django and FastAPI).

Managed by :

- Python software foundation

Community includes:

- Contributors
- Developers
- Educators

PART D : Open Source vs Proprietary Comparison

To fully contextualize Python's position, we must contrast it with a proprietary equivalent. MATLAB (Matrix Laboratory), developed by MathWorks, is an industry-standard proprietary language and environment used heavily in academia, engineering, and numerical computing.

Feature	**Python (Open Source)**	**MATLAB (Proprietary)**
Cost	Free (Zero licensing fees)	High (Expensive commercial and academic licenses)
Security/Auditing	Transparent (Anyone can audit source code)	Opaque (Closed-source, relies on vendor patching)
Flexibility	General-purpose (Web, AI, automation, networking)	Domain-specific (Primarily mathematics and engineering)
Control	Community-driven (PSF, PEP consensus)	Corporate-controlled (MathWorks dictates features)

Modification Freedom	Absolute (Can fork and rewrite the interpreter)	None (Cannot legally view or modify core engine)

Cost and Security:

Engineering teams are forced to pay ongoing licensing fees due to MATLAB's high financial barrier to entry. This access is made more accessible by Python. Additionally, "Linus's Law" ("given enough eyeballs, all bugs are shallow") is advantageous to Python. While MATLAB depends solely on MathWorks' internal teams to find and fix vulnerabilities, Python's open nature enables international security researchers to audit the code.

Flexibility and Control :

MATLAB performs poorly outside of its specialized mathematical modeling domain. Because Python is a general-purpose language, a developer can use it to write the infrastructure deployment scripts, train a machine learning model, and create a REST API to serve it. While corporate profitability metrics drive MATLAB's roadmap, public needs drive Python's evolution through the PEP process.\

Final Verdict and Recommendation :

According to this audit, Python clearly shows that the open-source model is better for general-purpose computing. In highly specialized, legacy engineering environments where stringent vendor support is required, MATLAB's proprietary model is still relevant. However, Python is unquestionably the superior option for contemporary software engineering, data science, and artificial intelligence.

I strongly suggest Python as a foundational language for any technical curriculum, not just as a tool. In addition, I would definitely contribute to its ecosystem. Giving back to the ecosystem—whether by improving documentation, submitting bug reports to data-processing libraries, or open-sourcing my own automation tools—is both a professional obligation and a competitive advantage. I have witnessed the power of community-driven development while developing autonomous AI agents and using web scraping frameworks in hackathon environments.

Python is better for:

- Students
- Developers
- Startups

2. Conclusion

From its philosophical roots to its actual implementation on Linux file systems, this audit has examined the Python programming language. As we've seen, Guido van Rossum's focus on readability gave rise to a language that, when combined with the permissive PSF license, created one of the most dynamic intellectual ecosystems in human history. Compared to proprietary systems like MATLAB, open source eliminates financial barriers, speeds up innovation, and transfers authority from corporations to communities.

When one considers the significance of open-source software for a developer's career, it becomes evident that FOSS is a continuous, international mentorship program rather than just a development methodology. A developer can learn from the industry's collective brilliance by working with open-source codebases, reading implementation details of intricate algorithms, and using shared dependencies to create quick prototypes. Python is proof that software can truly transform the world when it is developed transparently, shared freely, and governed ethically.